



JAVASCRIPT

INTERVIEW QUESTIONS



www.w3tec.com



info@w3tec.com



+91 74169 39333



Question 1

What will be the output of the following JavaScript code?

```
let num = 5;  
console.log(typeof typeof num);
```

Answer

The output will be "string".

Explanation:

- The output is "string" because `typeof num` returns 'number', and `typeof 'number'` returns 'string'

Question 2

What will be the output of the following JavaScript code?

```
let a = 1;  
let b = new Number(1);  
console.log(a == b);  
console.log(a === b);
```

Answer

The output will be "true" and "false"

Explanation:

- `a == b` is true due to type coercion, while `a === b` is false because strict equality checks both value and type.

Question 3

What will be the output of the following JavaScript code?

```
name = "w3 technologies";  
console.log(name);  
let name;
```

Answer

The code will throw a ReferenceError.

Explanation:

- Accessing name before its declaration with let results in a ReferenceError due to the temporal dead zone (TDZ).

Question 4

What will be the output of the following JavaScript code?

```
let name = "web wide winners";  
const institute = {  
  name: "Technologies",  
  fees: 100,  
  name,  
};  
console.log(institute.name);
```

Answer

The output will be: "web wide winners"

Explanation:

- When creating the institute object, name is included as a shorthand property, which overrides the previously declared "Technologies" key-value pair. Hence, institute.name refers to the value of the variable name, which is "web wide winners".

Question 5

What will be the output of the following JavaScript code?

```
const numbers = [1, 2, 3, 4];  
const [y] = numbers;  
console.log(4);
```

Answer

The output will be: 4

Explanation:

- The code destructures the first element of the numbers array into the variable y, but the console.log(4) statement explicitly logs the number 4 and is unrelated to the destructuring..

Question 6

What will be the output of the following JavaScript code?

```
const arr = [1, 2, 3, 4];  
const mapped = arr.flatMap([x => x, x * 2]);  
console.log(mapped);
```

Answer

The code will throw a SyntaxError..

Explanation:

- The flatMap() method expects a single callback function as its argument. However, [x => x, x * 2] is an array, not a valid function. This causes a syntax error in the code. To fix it, provide a proper callback function, e.g., arr.flatMap(x => [x, x * 2]);.

Question 7

What will be the output of the following JavaScript code?

```
const numbers = [1, 2, 3, 4];  
numbers.reverse();  
numbers.push(6);  
console.log(numbers);
```

Answer

The output will be: [6, 4, 3, 2, 1]

Explanation:

- The `reverse()` method reverses the order of elements in the numbers array, changing it to [4, 3, 2, 1].
- The `push(6)` method adds 6 to the end of the array, resulting in [6, 4, 3, 2, 1].

Question 8

What will be the output of the following JavaScript code?

```
const movies = ["devara", "pushpa"];  
movies.sort((a, b) => b.localeCompare(a));  
console.log(movies);
```

Answer

The output will be: ["pushpa", "devara"]

Explanation:

- The `sort()` method sorts the array in descending order using `localeCompare()`, so the result is ["pushpa", "devara"].

Question 9

What will be the output of the following JavaScript code?

```
console.log("start");
setTimeout(() => {
  console.log("timeout");
}, 0);
Promise.resolve().then(() => {
  console.log("Promise");
});
console.log("End");
```

Answer

The output will be:

sql
startEnd
Promise
timeout

Explanation:

- `console.log("start")` runs first.
- `setTimeout` is placed in the event queue with a delay of 0, so it waits until the call stack is empty.
- `Promise.resolve().then()` is part of the microtask queue and gets executed after the synchronous code.
- `console.log("End")` runs next as part of the synchronous code.
- Finally, the microtask `Promise` runs before the `setTimeout` callback, so "Promise" is logged before "timeout".

Question 10

What will be the output of the following JavaScript code?

```
var obj = {  
  a: "hello world",  
  b: 42  
};  
var b = "a";  
console.log(obj[b]);  
console.log(["b"]);
```

Answer

The output will be:

```
hello world  
["b"]
```

Explanation:

- `obj[b]` accesses the property of `obj` using the value of `b`, which is `"a"`, so `obj["a"]` results in `"hello world"`.
- `console.log(["b"])` logs the array `["b"]` as it is, not the value of the property `b` in the `obj`.

Question 11

What is currying in JavaScript, and how does it work? Provide an example to demonstrate its usage.?

Answer

Currying is a technique where a function taking multiple arguments is transformed into a series of functions, each accepting one argument, enabling partial application.

Example:

```
.function add(a) {  
  return function(b) {  
    return a + b;  
  };  
}  
  
const add5 = add(5); // Fixes `a` to 5  
console.log(add5(3)); // Outputs 8
```

Question 12

What is a prototype in JavaScript, and how does it enable inheritance between objects?

Answer

.In JavaScript, a prototype is an object that allows objects to inherit properties and methods from a constructor, forming a prototype chain.

Example:

```
function Person(name) {  
  this.name = name;  
}  
Person.prototype.greet = function() {  
  console.log(`Hello, my name is ${this.name}`);  
};  
const person1 = new Person("Alice");  
person1.greet(); // Outputs: "Hello, my name is Alice"
```

Question 13

What are JavaScript modules and object modules, and how do they work?

Answer

JavaScript modules allow you to split code into reusable units, enabling better organization and avoiding global scope pollution. You can export variables, functions, or classes from one module and import them into others.

Object modules refer to structuring code using objects, where each object can serve as a module with its own methods and properties.

Example:

```
math.js(module)  
export function add(a, b) {  
  return a + b;  
}  
main.js:  
import { add } from './math.js';  
console.log(add(2, 3)); // Outputs 5
```

In object modules, you can export an entire object with methods and properties:

```
// person.js
export const person = {
  name: 'Alice',
  greet() {
    console.log(`Hello, ${this.name}`);
  }
};
```

Question 14

What is the difference between synchronous and asynchronous JavaScript, and how do they work?

Answer

Synchronous JavaScript executes code sequentially, meaning one task completes before the next starts. It blocks the execution of further code until the current task finishes

Asynchronous JavaScript allows tasks to run in the background without blocking the execution of other code. This is often used for operations like file reading, network requests, or timers.

Example:

Example of Synchronous Code:

```
console.log("Start");
console.log("Middle");
console.log("End");
// Outputs: Start, Middle, End
```

Example of Asynchronous Code:

```
console.log("Start");
setTimeout(() => {
  console.log("Middle");
}, 1000);
console.log("End");
// Outputs: Start, End, Middle (after 1 second)
```

Question 15

What is the difference between the spread operator and the rest operator in JavaScript?

Answer

- Spread Operator (...): Expands elements of an array or object into individual elements.

Example:

```
const arr = [1, 2, 3];  
const newArr = [...arr, 4]; // [1, 2, 3, 4]  
const obj = { a: 1, b: 2 };  
const newObj = { ...obj, c: 3 }; // { a: 1, b: 2, c: 3 }
```

- Rest Operator (...): Collects multiple elements into an array or object.

Example:

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b);  
}  
console.log(sum(1, 2, 3)); // 6
```

```
const { a, ...rest } = { a: 1, b: 2, c: 3 };  
console.log(rest); // { b: 2, c: 3 }
```

Question 16

What is the difference between event bubbling and event capturing in JavaScript?

Answer

- Event Bubbling: The event starts from the innermost target element and propagates upward to its ancestors.

Example:

```
document.getElementById('child').addEventListener('click', () => console.log('Child clicked'));  
document.getElementById('parent').addEventListener('click', () => console.log('Parent clicked'));  
// Clicking the child logs: "Child clicked" -> "Parent clicked"
```

Answer

- Event Capturing: The event starts from the outermost ancestor and propagates downward to the target element.

Example:

```
document.getElementById('parent').addEventListener('click', () => console.log('Parent clicked'), true);
document.getElementById('child').addEventListener('click', () => console.log('Child clicked'), true);
// Clicking the child logs: "Parent clicked" -> "Child clicked"
```

Question 17

What is the difference between a shallow copy and a deep copy in JavaScript?

Answer

Shallow Copy:

- Copies only the first layer of an object or array. Nested objects or arrays are still referenced, not copied.

Example:

```
const obj = { a: 1, b: { c: 2 } };
const shallowCopy = { ...obj };
shallowCopy.b.c = 3;
console.log(obj.b.c); // 3 (nested object is shared)
```

Deep Copy:

- Creates an entirely independent copy, including all nested objects or arrays. Changes to the copy do not affect the original.

Example:

```
const obj = { a: 1, b: { c: 2 } };
const deepCopy = JSON.parse(JSON.stringify(obj));
deepCopy.b.c = 3;
console.log(obj.b.c); // 2 (nested object is not shared)
```

Question 18

What are first-class functions in JavaScript?

Answer

First-class functions are functions that are treated as first-class citizens, meaning they can be:

- Assigned to variables:

```
const greet = function() {
  return "Hello!";
};
console.log(greet()); // "Hello!"
```

- Passed Arguments:

```
function sayHello(func) {
  console.log(func());
}
sayHello(() => "Hi!"); // "Hi!"
```

- Returned from other functions:

```
function createGreeter() {
  return function() {
    return "Hello, World!";
  };
}
const greeter = createGreeter();
console.log(greeter()); // "Hello, World!"
```

Question 19

What is destructuring in JavaScript, and how does it work?

Answer

Destructuring in JavaScript is a way to unpack values from arrays or properties from objects into separate variables. It makes your code cleaner and easier to read.

Example with an Array:

```
const fruits = ['apple', 'banana', 'cherry'];
const [first, second] = fruits;
console.log(first); // 'apple'
console.log(second); // 'banana'
```

Example with an object:

```
const user = { name: 'John', age: 30 };  
const { name, age } = user;  
console.log(name); // 'John'  
console.log(age); // 30
```

Question 20

How does the event loop work in JavaScript?

Answer

The event loop in JavaScript handles asynchronous code (like `setTimeout`, promises, and I/O operations) by managing the execution of tasks. Here's how it works:

1. Call Stack:

- JavaScript runs synchronous code line by line in the call stack.
- When it encounters asynchronous code, it sends it to other parts like the Web APIs (browser environment).

2. Web APIs / Node APIs:

- Asynchronous tasks (e.g., timers, fetch requests) are handled here until they're ready to complete.

3. Task Queue (or Callback Queue):

- Once an async operation finishes, its callback is added to the task queue.

4. Microtask Queue:

- Promises and other microtasks (like `MutationObserver`) go here.
- Microtasks have higher priority than tasks in the callback queue.

5. Event Loop:

- The event loop continuously checks if the call stack is empty.
- If the stack is clear, it pushes tasks from the microtask queue first, then the callback queue, into the call stack to be executed.

Visual Flow:

1. Call Stack → Empty?
2. Microtask Queue → Process all microtasks
3. Task Queue → Process one task
4. Repeat!

Example:

```
console.log('Start'); // Synchronous
setTimeout(() => console.log('Timeout'), 0); // Task Queue
Promise.resolve().then(() => console.log('Promise')); // Microtask Queue
console.log('End'); // Synchronous
```

OUTPUT:

Start
End
Promise
Timeout

Question 21

What is the difference between Local Storage and Session Storage in JavaScript?

Answer:

Both Local Storage and Session Storage are web storage APIs used to store data in the browser, but they differ in lifespan and scope.

Difference Between Local Storage and Session Storage:

1. Lifespan:

- Local Storage: Data persists even after the browser is closed or refreshed.
- Session Storage: Data is cleared when the browser tab is closed.

2. Scope:

- Local Storage: Data is shared across all tabs of the same browser for a website.
- Session Storage: Data is specific to the current tab only.

3. Storage Limit:

- Both provide around 5-10 MB of storage, but it depends on the browser.

4. Use Cases:

- Local Storage: Suitable for storing long-term data like user preferences or settings.
- Session Storage: Ideal for temporary data like session-related information or user progress in a single tab.

Example:

Local Storage:

```
localStorage.setItem('username', 'John'); // Save
console.log(localStorage.getItem('username')); // Retrieve: 'John'
localStorage.removeItem('username'); // Remove
```

Session Storage:

```
sessionStorage.setItem('cart', '2 items'); // Save  
console.log(sessionStorage.getItem('cart')); // Retrieve: '2 items'  
sessionStorage.removeItem('cart'); // Remove
```

Question 22

What is event delegation in JavaScript, and why is it useful?

Answer

Event delegation is a technique in JavaScript where you attach a single event listener to a parent element to handle events on its child elements. This works because events "bubble up" from the child to the parent.

Why is it useful?

1. Fewer Listeners: Instead of adding event listeners to every child, you use just one on the parent, which is more efficient.
2. Dynamic Elements: It works even for elements added later (dynamically).
3. Cleaner Code: Reduces complexity and makes code easier to manage.

Example:

Without event delegation (not efficient):

```
document.querySelectorAll('.btn').forEach(button => {  
  button.addEventListener('click', () => console.log('Button clicked'));  
});
```

With event delegation (better):

```
document.querySelector('#parent').addEventListener('click', event => {  
  if (event.target.classList.contains('btn')) {  
    console.log('Button clicked');  
  }  
});
```

Question 23

What is the difference between `setTimeout` and `setInterval` in JavaScript?

Answer

- `setTimeout`: Runs a function once after a delay.
- Example:

```
setTimeout(() => console.log('Runs once after 2 seconds'), 2000);
```
- `setInterval`: Runs a function repeatedly at regular intervals.

- Example:

```
const intervalId = setInterval(() => console.log('Runs every 2 seconds'), 2000);  
// To stop it:  
setTimeout(() => clearInterval(intervalId), 6000); // Stops after 6 seconds
```

Question 24

What are the differences between for, for...of, for...in, and forEach loops in JavaScript, and when should you use each?

Answer:

for Loop:

- Traditional loop for iterating over a range of numbers or array indices.
- You control the start, end, and step.
- Example:

```
for (let i = 0; i < 5; i++) {  
  console.log(i); // 0, 1, 2, 3, 4  
}
```

for...of Loop:

- Iterates over values of iterable objects like arrays, strings, or sets.
- Best for working with array elements.
- Example:

```
const fruits = ['apple', 'banana', 'cherry'];  
for (const fruit of fruits) {  
  console.log(fruit); // 'apple', 'banana', 'cherry'  
}
```

for...in Loop:

- Iterates over keys (properties) of an object or indices of an array.
- Best for objects but should avoid for arrays (use for...of instead).
- Example:

```
const user = { name: 'John', age: 30 };  
for (const key in user) {  
  console.log(key, user[key]); // 'name John', 'age 30'  
}
```

forEach Method:

- Calls a function for each element in an array.
- Best for executing functions on array elements.
- Cannot break or skip iterations.
- Example:

```
const numbers = [1, 2, 3];  
numbers.forEach(num => console.log(num)); // 1, 2, 3
```

Question 25

What is the difference between `call()`, `apply()`, and `bind()` methods in JavaScript?

Answer:

`call()` Method:

- Calls a function immediately with a specified this value and arguments provided individually.
- Example:

```
const person = { name: 'John' };  
function greet(greeting) {  
  console.log(`${greeting}, ${this.name}`);  
}  
greet.call(person, 'Hello'); // Output: 'Hello, John'
```

`apply()` Method:

- Similar to `call()`, but arguments are provided as an array.
- Example:

```
greet.apply(person, ['Hi']); // Output: 'Hi, John'
```

`bind()` Method:

- Returns a new function with a bound this value. The function can be called later.
- Example:

```
const boundGreet = greet.bind(person, 'Hey');  
boundGreet(); // Output: 'Hey, John'
```

Question 26

What is the difference between `forEach` and `for loop` in JavaScript?

Answer:

`for Loop`:

- A traditional loop that gives you full control over the iteration process (start, end, step).
- Can break (`break`) or skip (`continue`) iterations.
- Works for both arrays and other iterable objects.
- Example:

```
const numbers = [1, 2, 3, 4];  
for (let i = 0; i < numbers.length; i++) {  
  console.log(numbers[i]); // 1, 2, 3, 4  
}
```

forEach Method:

- A method specifically for arrays that executes a callback function for each element.
- Cannot use break or continue directly.
- Automatically handles the iteration process.
- Example:

```
numbers.forEach(num => {  
  console.log(num); // 1, 2, 3, 4  
});
```

Question 27

What is the difference between stable and unstable sorting algorithms in simple terms?

Answer:

Stable Sorting Algorithm:

- Keeps the original order of equal elements after sorting.
- Use it when the order of duplicate items matters.
- Examples: Bubble Sort, Merge Sort, Insertion Sort.
- Example:

Before sorting: [5a, 3, 5b]

After stable sorting: [3, 5a, 5b] (order of 5a and 5b is preserved).

Unstable Sorting Algorithm:

- Does not guarantee the original order of equal elements after sorting.
- Use it for faster performance if order doesn't matter.
- Examples: Quick Sort, Heap Sort, Selection Sort.
- Example:

Before sorting: [5a, 3, 5b]

After unstable sorting: [3, 5b, 5a] (order of 5a and 5b may change).

Question 28

How do you detect a cycle in a linked list?

Answer

To detect a cycle in a linked list, you can use the Floyd's Cycle Detection Algorithm (Two Pointer Method).

1. Use Two Pointers (Slow and Fast):

- Move one pointer (slow) one step at a time.
- Move the other pointer (fast) two steps at a time.

2. Check for a Cycle:

- If there's a cycle, the slow and fast pointers will eventually meet.
- If fast reaches the end (null), the list has no cycle.

Example:

```
function hasCycle(head) {  
  let slow = head;  
  let fast = head;  
  while (fast && fast.next) {  
    slow = slow.next;    // Move slow by 1 step  
    fast = fast.next.next; // Move fast by 2 steps  
    if (slow === fast) {  // Cycle detected  
      return true;  
    }  
  }  
  return false; // No cycle  
}
```

Question 29

What is the concept of cloning a graph, and how is it done?

Answer:

Cloning a graph means creating a deep copy of a graph, where every node and its connections (edges) are copied into a new graph without affecting the original graph.

Steps to Clone a Graph:

1. Understand the Graph Structure:

- A graph has nodes (or vertices) connected by edges.
- Each node typically has a value and a list of neighbors.

2. Use a Map to Track Nodes:

- Use a Map (or dictionary) to keep track of visited nodes and their clones to avoid re-cloning nodes.

3. Traverse the Graph:

- You can use Depth First Search (DFS) or Breadth First Search (BFS) to visit each node and its neighbors.
- For each node, create a new node, copy its value, and recursively clone its neighbors.

Code Example:

```
function cloneGraph(node) {  
  if (!node) return null;  
  const map = new Map(); // To store original-to-clone node mapping  
  const dfs = (current) => {  
    if (map.has(current)) return map.get(current);  
    const clone = new Node(current.val);  
    for (let neighbor of current.neighbors) {  
      clone.neighbors.push(dfs(neighbor));  
    }  
    map.set(current, clone);  
    return clone;  
  };  
  return dfs(node);  
}
```

```
const clone = { val: current.val, neighbors: [] };
map.set(current, clone);
for (const neighbor of current.neighbors) {
  clone.neighbors.push(dfs(neighbor));
}
return clone;
};
return dfs(node);
}
```

Question 30

What is a closure in JavaScript, and how can it be used to create private variables?

Answer:

Closure is a feature in JavaScript where a function "remembers" the variables from its outer scope even after the outer function has finished executing.

How Closures Work:

- A function can access variables defined in its parent scope.
- This behavior creates a "closure," where the inner function retains access to those variables.
- Creating Private Variables with Closures:

Closures are useful for creating private variables, which are variables that cannot be accessed directly from outside the function.

Example:

```
function createCounter() {
  let count = 0; // Private variable
  return {
    increment: () => count++, // Increments count
    getCount: () => count    // Returns count
  };
}
```

```
const counter = createCounter();
console.log(counter.getCount()); // Output: 0
counter.increment();
console.log(counter.getCount()); // Output: 1
```

Explanation:

- The count variable is private to the createCounter function.
- The returned increment and getCount methods form closures and retain access to count, even after createCounter finishes executing.
- This allows controlled access to count.

Question 31

What is backtracking in programming, and how does it work?

Answer:

Backtracking is a problem-solving technique used to explore all possible solutions by making choices and undoing them (backtracking) if they don't lead to a solution.

How It Works:

1. Start with a potential solution.
2. If it satisfies the problem, proceed further.
3. If it doesn't, go back (undo the last choice) and try another option.
4. Repeat until you find a solution or all possibilities are exhausted.

Common Use Cases:

- Solving puzzles (e.g., Sudoku, N-Queens).
- Generating permutations or combinations.
- Graph traversal problems (e.g., finding paths).

Example: Solving N-Queens Problem:

```
function solveNQueens(n) {  
  const board = Array.from({ length: n }, () => Array(n).fill('.'));  
  const results = [];  
  function isSafe(row, col) {  
    for (let i = 0; i < row; i++) {  
      if (board[i][col] === 'Q') return false;  
      if (col - (row - i) >= 0 && board[i][col - (row - i)] === 'Q') return false;  
      if (col + (row - i) < n && board[i][col + (row - i)] === 'Q') return false;  
    }  
    return true;  
  }  
  function backtrack(row) {  
    if (row === n) {  
      results.push(board.map(row => row.join(")));  
      return;  
    }  
  }  
}
```

```
for (let col = 0; col < n; col++) {  
  if (isSafe(row, col)) {  
    board[row][col] = 'Q';  
    backtrack(row + 1);  
    board[row][col] = '.'; // Undo choice  
  }  
}  
backtrack(0);  
return results;  
}  
console.log(solveNQueens(4));
```

Question 32

What is the difference between recursive and iterative algorithms?

Answer:

Recursive Algorithm:

- Solves a problem by calling itself with smaller subproblems.
- Requires a base case to stop the recursion.
- Uses the call stack to keep track of function calls.

Example: Factorial using recursion:

```
function factorial(n) {  
  if (n === 0) return 1; // Base case  
  return n * factorial(n - 1); // Recursive call  
}  
console.log(factorial(5)); // Output: 120
```

Iterative Algorithm:

- Solves a problem using loops (e.g., for, while).
- Does not use the call stack, so it avoids the overhead of recursive calls

Example: Factorial using iteration:

```
function factorial(n) {  
  let result = 1;  
  for (let i = 1; i <= n; i++) {  
    result *= i;  
  }  
}
```

```
return result;
}
console.log(factorial(5)); // Output: 120
```

Question 33

What are higher-order functions in JavaScript, and how are they used?

Answer:

A higher-order function is a function that either:

1. Takes another function as an argument, or
2. Returns a function as its result

Examples:

1. Taking a Function as an Argument:

- Commonly used with array methods like map, filter, and reduce.

```
const numbers = [1, 2, 3, 4];
// Using a higher-order function
const doubled = numbers.map(num => num * 2);
console.log(doubled); // Output: [2, 4, 6, 8]
```

Returning a Function:

- Often used for creating reusable functions (e.g., closures).

```
function multiplyBy(factor) {
  return num => num * factor;
}
const double = multiplyBy(2);
console.log(double(5)); // Output: 10
```

Question 34

What are higher-order functions in JavaScript, and how are they used?

Answer:

How They Work:

1. async Keyword:

- Declares a function as asynchronous.
- It automatically wraps the return value in a Promise.

2 .await Keyword:

- Pauses the execution of an async function until the promise resolves or rejects.
- It makes asynchronous code look and behave like synchronous code.

Example:

```
async function fetchData() {  
  try {  
    const response = await fetch('https://api.example.com/data');  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error('Error:', error);  
  }  
}  
  
fetchData();
```

Question 35

What are cookies in JavaScript, and how are they used?

Answer:

Cookies are small pieces of data stored in the browser to remember information about a user, such as preferences, login sessions, or tracking data.

Key Features of Cookies:

- Stored as key-value pairs in the browser.
- Sent automatically with every HTTP request to the server.
- Have attributes like expiration time, domain, and path to control their behavior.

How to Use Cookies in JavaScript:

- Set a Cookie:

```
document.cookie = "username=John; expires=Fri, 31 Dec 2024 23:59:59 GMT; path=/";
```

- Get a Cookie:

```
console.log(document.cookie); // Output: "username=John"
```

- Delete a Cookie:

```
document.cookie = "username=John; expires=Thu, 01 Jan 1970 00:00:00 GMT; path=/";
```

Common Uses of Cookies:

- User Authentication: Keep users logged in.
- Preferences: Save user settings (e.g., dark mode).
- Tracking: Track user behavior across sessions (analytics or ads).

Question 36

What are the advantages and disadvantages of monolithic and microservice architectures?

Answer:

Monolithic Architecture

In a monolithic architecture, the entire application is built as a single unit, with all components tightly integrated and deployed together.

Advantages:

1. **Simpler to Develop:** Easier to start with, as all components are in one place.
2. **Easy to Deploy:** One single deployment for the entire application.
3. **Performance:** Direct communication between components without network overhead.

Disadvantages:

1. **Scalability Issues:** Hard to scale specific components independently.
2. **Maintenance:** As the application grows, it becomes harder to manage and update.
3. **Flexibility Limitations:** Difficult to use different technologies for different parts of the application.

Microservice Architecture

In microservice architecture, the application is divided into smaller, independent services, each responsible for a specific business function.

Advantages:

1. **Scalability:** Individual services can be scaled independently based on load.
2. **Flexibility:** You can use different technologies for different services.
3. **Resilience:** If one service fails, the others can still function, making the system more fault-tolerant.
4. **Faster Development:** Teams can work on different services simultaneously without conflicts.

Disadvantages:

1. **Complexity:** Managing multiple services can be complicated.
2. **Communication Overhead:** Services communicate over a network, which can lead to latency issues.
3. **Deployment Complexity:** Requires tools for service discovery, load balancing, and orchestration (e.g., Kubernetes).
4. **Data Consistency:** Ensuring data consistency across services can be challenging.

Question 37

What is the role of deferred scripts in JavaScript?

Answer:

Deferred scripts are JavaScript files that are loaded and executed after the HTML document has been fully parsed, but before the DOMContentLoaded event is triggered.

How It Works:

- The defer attribute is added to the <script> tag.
- The browser will continue parsing the HTML while the script is being downloaded.
- Once the HTML is fully parsed, the deferred scripts are executed in the order they appear in the document.

Example:

```
<script src="script.js" defer></script>
```

Benefits of Deferred Scripts:

1. Faster Page Loading: Scripts don't block HTML parsing, improving the page load time.
2. Order Preservation: Scripts are executed in the order they appear in the HTML, ensuring dependencies are respected.

Question 38

What is the difference between ES6 and ES5 in JavaScript?

Answer:

ES6 (ECMAScript 2015) introduced many new features and improvements to JavaScript compared to ES5 (ECMAScript 2009). Here are the main differences:

1. Variable Declarations:

- ES6 introduced let and const for block-scoped variables, offering better control over variable scope.
- ES5 only had var, which has function-level scope, leading to potential issues with variable hoisting.

2. Functions:

- ES6 introduced arrow functions, which offer a shorter syntax and lexically bind the this keyword, making them easier to use in many cases.
- ES5 used regular function expressions that can sometimes lead to confusing this behavior.

3. String Interpolation:

- ES6 introduced template literals, allowing easier string interpolation and multiline strings.
- ES5 required string concatenation for combining variables with text.

- Object-Oriented Programming:
 - ES6 introduced class syntax, making object-oriented programming more intuitive and cleaner.
 - ES5 used constructor functions and prototypes, which were more verbose and less intuitive.
- Modules:
 - ES6 added native support for modules using import and export, making it easier to organize and manage large codebases.
 - ES5 had no native module system, so developers had to rely on third-party libraries like RequireJS.
- Asynchronous Programming:
 - ES6 introduced Promise, providing a cleaner and more manageable way to handle asynchronous operations.
 - ES5 relied on callback functions, which could lead to callback hell and difficult-to-maintain code

Question 39

What is the nullish coalescing operator in JavaScript?

Answer:

The nullish coalescing operator (??) is used to provide a default value when a variable is null or undefined.

How It Works:

- If the value on the left is null or undefined, the operator returns the value on the right.
- If the value on the left is any other value (including falsy values like 0, NaN, or ""), it returns the left value.

Example:

```
let name = null;  
let defaultName = "Guest";  
console.log(name ?? defaultName); // Output: "Guest"
```

Question 40

What is an Immediately Invoked Function Expression (IIFE) in JavaScript?

Answer:

An Immediately Invoked Function Expression (IIFE) is a function that is defined and executed immediately after being created. It helps to create a private scope, preventing variables from polluting the global scope.

How It Works:

- An IIFE is defined inside parentheses and is executed right away by adding another set of parentheses at the end.
- Example:

```
(function() {  
  console.log("This is an IIFE!");  
})();
```

Question 41

What is the Temporal Dead Zone (TDZ) in ES6?

Answer:

The Temporal Dead Zone (TDZ) refers to the period between the creation of a variable (using `let` or `const`) and its initialization, during which any reference to that variable will throw a `ReferenceError`.

How It Works:

- In ES6, variables declared with `let` or `const` are hoisted to the top of their scope, but they cannot be accessed until they are initialized. During this period, they are in the TDZ.
- Accessing these variables before initialization results in an error.

Example:

```
console.log(myVar); // ReferenceError: Cannot access 'myVar' before initialization  
let myVar = 5;
```

Why It Happens:

- Hoisting moves the declaration to the top, but the variable is not initialized until the line where the assignment happens.
- The TDZ ensures that you don't accidentally use a variable before it's ready, improving code reliability and reducing bugs.

Question 42

What is the Prototype Design Pattern in JavaScript?

Answer:

The Prototype Design Pattern is a creational pattern that allows you to create new objects by copying an existing object, known as the prototype. It is useful when you want to create a new object based on an existing one, instead of creating a new instance from scratch.

How It Works:

- In JavaScript, each object has a prototype property, which points to another object. This allows objects to inherit properties and methods from their prototype.
- The Prototype Pattern creates a copy of an existing object and then allows you to modify the new copy as needed.

Example:

```
// Original prototype
const carPrototype = {
  drive() {
    console.log("Driving...");
  },
  stop() {
    console.log("Stopping...");
  }
};

// Create a new car based on the prototype
const car1 = Object.create(carPrototype);
car1.drive(); // Output: "Driving..."

// Modify the new car object
car1.color = "red";
console.log(car1.color); // Output: "red"
```

Question 43

What is a generator in JavaScript?

Answer:

A generator in JavaScript is a special type of function that can pause its execution and later resume it. It allows you to generate a sequence of values over time, instead of computing them all at once and returning them in an array.

Key Features:

1. Generator Function: A function that can pause its execution using the yield keyword and resume from where it left off.
2. Iterator: A generator function returns an iterator, which can be used to retrieve values one by one.

How It Works:

- You define a generator function using the function* syntax.
- Inside the function, you use the yield keyword to pause execution and return a value.
- Each time the generator is called, it resumes from where it last yielded a value.

Example:

```
function* myGenerator() {  
  yield 1;  
  yield 2;  
  yield 3;  
}  
  
const gen = myGenerator();  
console.log(gen.next().value); // Output: 1  
console.log(gen.next().value); // Output: 2  
console.log(gen.next().value); // Output: 3
```

Question 44

Where are anonymous functions used in JavaScript?

Answer:

Anonymous functions in JavaScript are functions that are defined without a name. They are commonly used in situations where a function is needed temporarily and does not need to be reused or referenced by name.

Common Use Cases for Anonymous Functions:

- Callbacks:

Anonymous functions are often used as callbacks in functions like `setTimeout`, `setInterval`, and event listeners.

```
setTimeout(function() {  
  console.log("This message is displayed after 2 seconds");  
}, 2000);
```

- Event Listeners:

When handling events like clicks or key presses, anonymous functions are used directly inside event listener methods

```
document.getElementById("myButton").addEventListener("click", function() {  
    alert("Button clicked!");  
});
```

- Array Methods:

Methods like map, filter, reduce, and others often use anonymous functions to process elements of an array.

```
const numbers = [1, 2, 3];  
const squaredNumbers = numbers.map(function(num) {  
    return num * num;  
});
```

- Immediately Invoked Function Expressions (IIFE):

Anonymous functions are used in IIFEs, which are functions that run as soon as they are defined.

```
(function() {  
    console.log("IIFE executed");  
})();
```

- Functional Programming:

In functional programming paradigms, anonymous functions are used to pass behavior (functions) as arguments to other functions.

```
function processData(data, callback) {  
    callback(data);  
}  
  
processData([1, 2, 3], function(arr) {  
    return arr.map(x => x * 2);  
});
```

Question 45

What are self-invoking functions in JavaScript?

Answer:

A self-invoking function, also known as an Immediately Invoked Function Expression (IIFE), is a function that is defined and executed immediately after being created, without being explicitly called.

How It Works:

- It is an anonymous function wrapped in parentheses, followed by another pair of parentheses to invoke it immediately.
- This pattern is used to run a function as soon as it is defined.

Example:

```
(function() {  
  console.log("This is a self-invoking function!");  
})();
```

Why Use Self-Invoking Functions?

1. Encapsulation: It creates a private scope, meaning variables inside the function are not accessible from the global scope, helping to avoid polluting the global namespace.
2. Initialization: Useful for initialization code that should run only once, such as setting up event listeners or configuration settings.
3. Avoiding Global Variables: Since variables inside the function are scoped locally, it prevents unintentional conflicts with global variables.

Question 46

What is variable shadowing in JavaScript?

Answer:

Variable shadowing in JavaScript occurs when a variable declared inside a certain scope (such as within a function or block) has the same name as a variable declared in an outer scope, effectively "shadowing" or hiding the outer variable within the inner scope.

How It Works:

- The inner variable takes precedence over the outer variable, meaning when you reference the variable inside the inner scope, it refers to the inner variable, not the one in the outer scope.
- The outer variable remains unchanged outside the inner scope, but it becomes inaccessible while inside the inner scope.

```
let x = 10; // Outer variable  
function myFunction() {  
  let x = 20; // Inner variable shadows the outer one  
  console.log(x); // Output: 20 (refers to the inner variable)  
}  
myFunction();  
console.log(x); // Output: 10 (refers to the outer variable)
```

Question 47

What is strict mode in JavaScript?

Answer:

Strict mode in JavaScript is a way to opt into a stricter set of rules for writing code. It helps catch common programming errors and prevents certain features that are considered bad practice or error-prone. Strict mode can be enabled by adding "use strict"; at the beginning of a script or a function.

Key Features of Strict Mode:

1. Eliminates Silent Errors:
2. JavaScript in strict mode throws errors for actions that would otherwise fail silently, like assigning a value to an undeclared variable.
3. Prevents the Use of Certain Keywords:
4. It disallows the use of some keywords (like eval, arguments, etc.) to avoid confusion and reduce errors in the code.
5. Makes Assignment More Secure:
6. In strict mode, assigning a value to a non-writable property or a read-only object will throw an error, making the code more predictable.
7. Disallows this in Global Context:
8. In strict mode, if you reference this in a function without an explicit object context (like in a regular function), it will be undefined, not the global object (like window in browsers).
9. Prevents Duplicate Parameter Names:
10. It disallows function parameters to have duplicate names, which helps avoid ambiguity.

How to Use Strict Mode:

- Globally:

```
"use strict";
```

```
// The entire script will run in strict mode
```

- Locally (Inside a Function):

```
function myFunction() {
```

```
  "use strict";
```

```
  // Only this function will run in strict mode
```

```
}
```

Question 48

How does the garbage collector work in JavaScript?

Answer:

The garbage collector in JavaScript is an automatic memory management feature that reclaims memory used by objects that are no longer needed, helping to prevent memory leaks. It works behind the scenes to identify and remove objects that are no longer accessible, freeing up memory for new data.

How It Works:

1. **Mark-and-Sweep Algorithm:** JavaScript's garbage collection mainly uses a mark-and-sweep algorithm. Here's how it works:
 - **Marking:** The garbage collector starts by marking all objects that are still reachable (i.e., objects that can be accessed by the code). This includes objects that are directly or indirectly referenced from global variables, active functions, or other objects in memory.
 - **Sweeping:** After marking the reachable objects, the garbage collector sweeps through memory and removes all unmarked objects (those that are no longer accessible or referenced).
2. **Reachability:**
3. An object is considered reachable if it can be accessed through any reference from the root (like global variables, function parameters, or properties of other objects). Once an object becomes unreachable, it is eligible for garbage collection.
4. **Reference Counting (older approach):**
 - Earlier JavaScript engines also used reference counting, where objects were removed when the reference count (i.e., the number of references pointing to an object) reached zero. However, this method was less reliable because it couldn't handle circular references (when two objects reference each other, forming a cycle).
5. **Generational Garbage Collection (modern engines):**
 - Modern JavaScript engines use a generational garbage collection strategy. They divide objects into different "generations" (young, old) based on their age. New objects are allocated in the young generation, and objects that survive several garbage collection cycles are promoted to the old generation. This improves efficiency by focusing on cleaning up the young generation, where most objects are short-lived.

Example:

```
function createObject() {  
  let obj = { name: "example" };  
  // obj is reachable and will not be garbage collected  
  return obj;  
}
```

```
let obj1 = createObject();  
// At this point, obj1 is reachable and not eligible for garbage collection.
```

```
obj1 = null; // Now, obj1 is no longer reachable, so it is eligible for garbage collection.
```

Question 49

What is hoisting in JavaScript?

Answer:

Hoisting is a behavior in JavaScript where variable and function declarations are moved to the top of their containing scope during the compile phase, before the code has actually been executed. This means that you can refer to variables and functions before they are declared in the code.

How It Works:

- Variables:

For variables declared with `var`, the declaration is hoisted, but the assignment remains in place. This results in the variable being undefined until the assignment is executed.

```
console.log(x); // Output: undefined (hoisted but not initialized)  
var x = 5;  
console.log(x); // Output: 5
```

- Functions:

Function declarations are fully hoisted, meaning both the function definition and its body are moved to the top. Therefore, you can call a function before it's declared.

```
myFunction(); // Output: "Hello!"  
function myFunction() {  
  console.log("Hello!");  
}
```

Example:

```
function createObject() {  
  let obj = { name: "example" };  
  // obj is reachable and will not be garbage collected  
  return obj;  
}
```

```
let obj1 = createObject();  
// At this point, obj1 is reachable and not eligible for garbage collection.
```

```
obj1 = null; // Now, obj1 is no longer reachable, so it is eligible for garbage collection.
```

Question 50

What are the different types of errors in JavaScript?

Answer:

JavaScript has several types of errors that occur during development:

- Syntax Errors:

```
console.log("Hello" // SyntaxError: Unexpected end of input
```

These occur when the code violates the JavaScript syntax rules (e.g., missing parentheses or braces).

- Reference Errors:

These happen when you try to access a variable or function that doesn't exist.

```
console.log(x); // ReferenceError: x is not defined
```

- Type Errors:

These occur when an operation or function is applied to a value of the wrong type.

```
let num = 5;
```

```
num.toUpperCase(); // TypeError: num.toUpperCase is not a function
```

- Range Errors:

These happen when a value is outside the allowable range (e.g., exceeding the number of array elements).

```
let arr = new Array(-1); // RangeError: Invalid array length
```

- Eval Errors:

These are related to the misuse of the `eval()` function, though they are rarely encountered in modern JavaScript.

// EvalError example (rarely occurs in modern code)

- URI Errors:

These occur when there's an issue with encoding or decoding a URI (e.g., invalid characters).

`decodeURIComponent('%');` // URIError: URI malformed

Thank You for Choosing **W3 Technologies**

Thank you for choosing to explore this course from W3 Technologies. We hope you find it insightful and valuable as you embark on your learning journey.

At W3 Technologies, we believe that learning is a continuous process, and we are excited to be part of your journey towards gaining new skills and knowledge.



www.w3tec.com



info@w3tec.com



+91 74169 39333



**4th Floor, LR Shine Building, Vrc,
above IDBI Bank, beside 1-Town
Police Station, Somasekara
Puram, Nellore, Andhra Pradesh
524001.**

